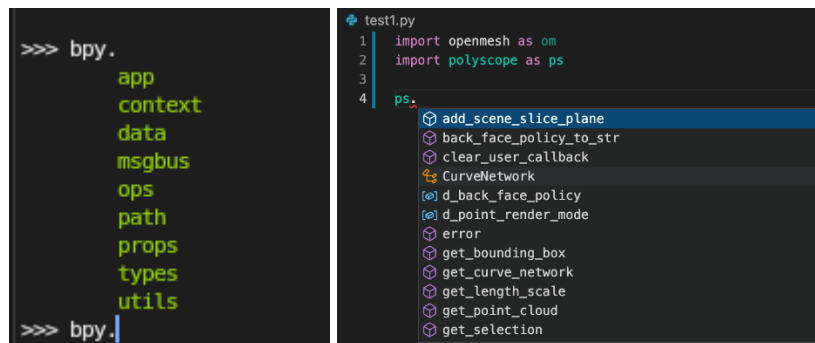


Relatório 07 – Se aprofundando no Blender Python

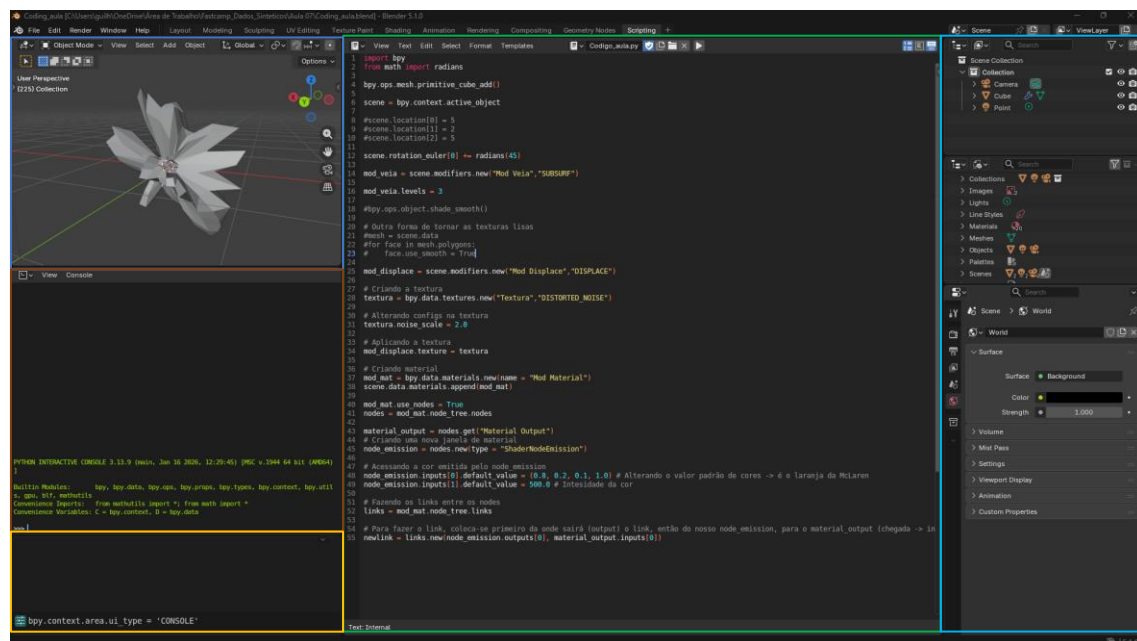
Guilherme Loan Schneider

Descrição da atividade

Primeira coisa que me chamou atenção utilizando o editor de texto do Blender, é a falta do complete do Python, onde escrever “bpy.” já apareceria as opções possíveis para escrever após esse ponto, como um autocomplete. No Blender a opção que temos é clicando na tecla TAB no console, que lista todas as possibilidades. Funciona de forma análoga ao Python no terminal do Windows.



A aula abordou vários aspectos da utilização do Python no Blender. A primeira questão foi uma rápida introdução a interface de Scripting do Blender. Na Figura abaixo temos várias telas diferentes para acompanhar. Em azul temos a visualização tradicional do que estamos criando, logo abaixo, em vermelho, temos o console do Python, onde alguns imports já são feitos por padrão. Em amarelo, um espaço reservado para mostrar tudo que é feito no Blender, como se fosse um “histórico”. No meio, em verde, temos o editor de texto utilizado por nós para criar os scripts. À Direita, em ciano, temos o painel tradicional do blender, com os objetos da cena, configurações do mundo, objetos, dentre outros.



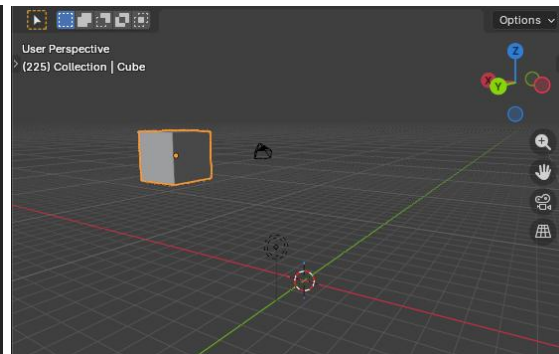
Começamos com algo simples, que foi adicionar um objeto na cena, mais especificamente, um cubo. Após isso, aplicamos alterações de posição nos eixos X, Y e Z. Note que os imports devem sempre serem feitos no editor de texto do Blender, no console não é necessário importar.

```
import bpy
from math import radians

bpy.ops.mesh.primitive_cube_add()

scene = bpy.context.active_object

scene.location[0] = 5
scene.location[1] = 2
scene.location[2] = 5
```



Em seguida, aplicamos uma rotação no cubo da cena, onde precisamos da biblioteca radians para efetuar a rotação (ou se preferir fazer o cálculo manualmente numa variável também dá). Note que a rotação pode ser nos três eixos, no caso abaixo, a rotação é aplicada no eixo X (Primeira posição do vetor).

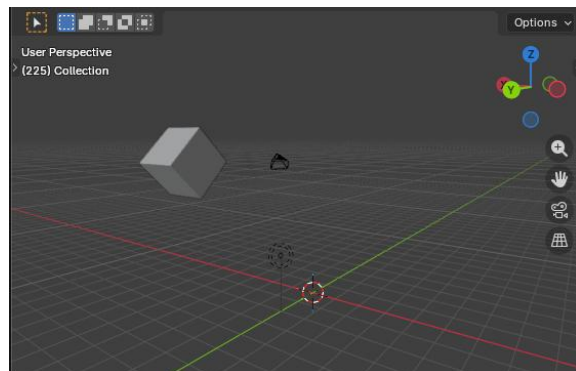
```
import bpy
from math import radians

bpy.ops.mesh.primitive_cube_add()

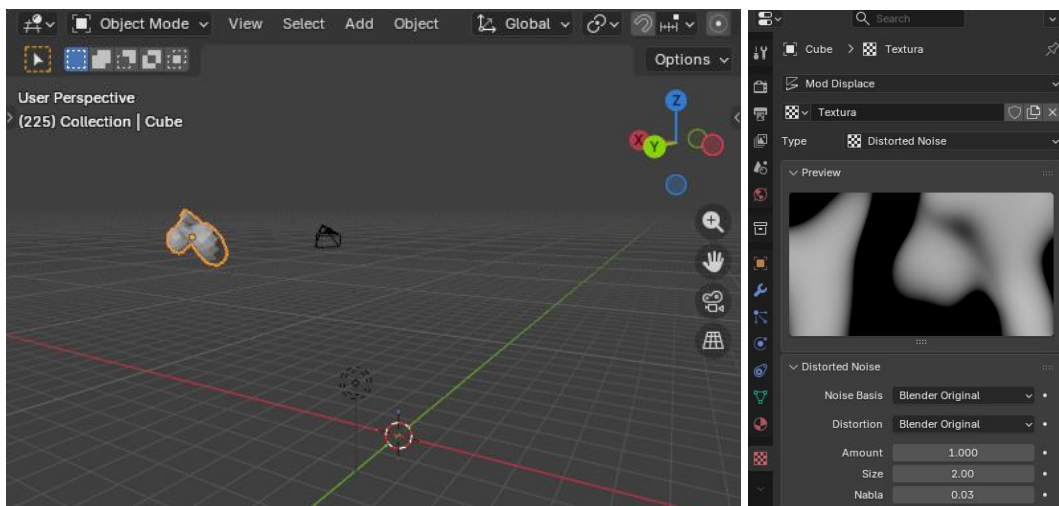
scene = bpy.context.active_object

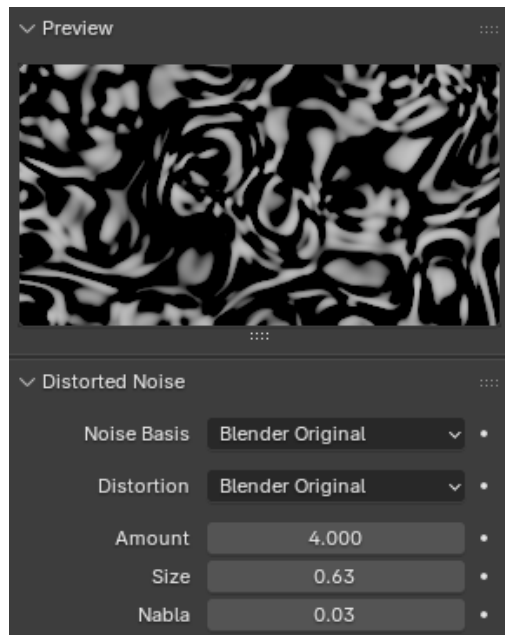
scene.location[0] = 5
scene.location[1] = 2
scene.location[2] = 5

scene.rotation_euler[0] += radians(45)
```

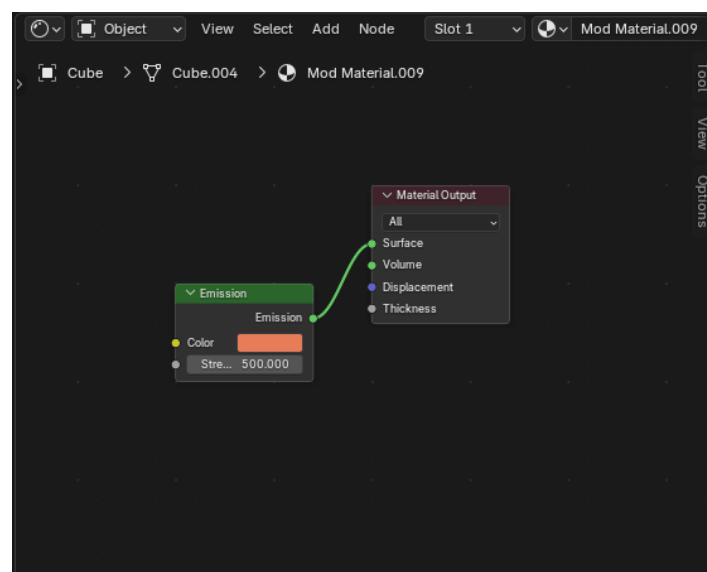


Passamos agora para os modificadores, texturas e materiais. Os modificadores permitem a manipulação dos objetos na cena, sem alterar a estrutura original. As texturas são aplicadas, em alguns casos, com modificadores, a fim de gerar algum comportamento em um objeto. No caso da aula, utiliza-se o “DISTORTED_NOISE”, que adiciona um ruído distorcido e sem muito sentido. Nós conseguimos alterar o comportamento do ruído a partir dos parâmetros na Figura à direita abaixo.

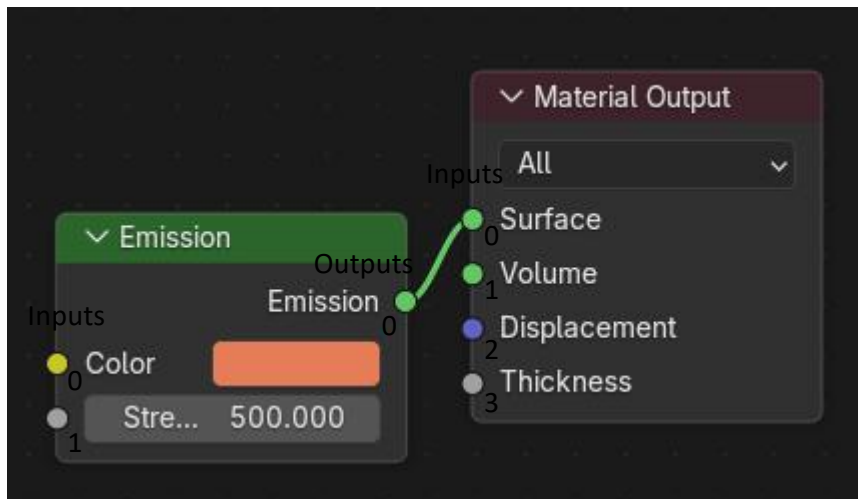




O material, por sua vez, permitirá que manipulemos várias informações do objeto na cena renderizada como cor, brilho, transparência, reflexão e até mesmo a forma como a luz interage com a superfície. No Blender, os materiais são configurados principalmente através do Shader Editor.



Na Figura acima, podemos verificar que existe uma conexão entre cada nó, essa conexão foi feita através de código em Python de forma até bem simples. Cada nó possui suas Inputs (bolinhas no lado esquerdo) e Outputs (bolinhas no lado direito), quando queremos efetuar uma conexão entre eles, utilizamos apenas uma linha de código, "newlink = links.new(output do node 1, input do node 2)". Note que as inputs e outputs funcionam como um vetor de posições (A Figura abaixo ilustra essa ideia).



Por fim, o autor demonstrou a criação de um “Operator Simple”, que funciona como uma forma do usuário criar uma função que executa tudo aquilo que ele escreveu no editor de texto do Blender. Além disso, ele permite que você crie um objeto dentro do Blender que, ao clicar F3 e procurar pelo nome que você criou, será feita a execução de tudo aquilo que você colocou na função execute do arquivo do Operator. A Figura abaixo ilustra um arquivo padrão de um Operator Simple.

```
import bpy

def main(context):
    for ob in context.scene.objects:
        print(ob)

class SimpleOperator(bpy.types.Operator):
    """Tooltip"""
    bl_idname = "object.simple_operator"
    bl_label = "Simple Object Operator"

    @classmethod
    def poll(cls, context):
        return context.active_object is not None

    def execute(self, context):
        main(context)
        return {'FINISHED'}

def menu_func(self, context):
    self.layout.operator(SimpleOperator.bl_idname, text=SimpleOperator.bl_label)

# Register and add to the "object" menu (required to also use F3 search "Simple O
def register():
    bpy.utils.register_class(SimpleOperator)
    bpy.types.VIEW3D_MT_object.append(menu_func)

def unregister():
    bpy.utils.unregister_class(SimpleOperator)
    bpy.types.VIEW3D_MT_object.remove(menu_func)

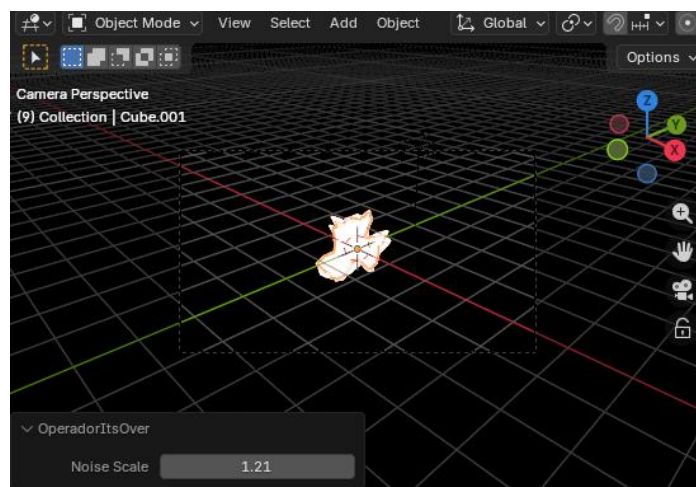
if __name__ == "__main__":
    register()

    # Test call.
    bpy.ops.object.simple_operator()
```

Esse arquivo permite também criar algumas funções personalizadas para alterar algum valor de objetos em uma cena. A Figura abaixo ilustra a utilização de uma propriedade do tipo Float para alterar o valor de ruído do objeto criado por nós durante a aula.

```
# Criando propriedades
noise_scale : FloatProperty(
    name = "Noise Scale",
    description = "Escala do ruído",
    default = 1.0,
    min = 0.0,
    max = 4.0
)
```

Na Figura abaixo conseguimos ver a propriedade no canto inferior esquerdo da tela, permitindo que alteremos o valor de Noise da textura aplicada no objeto.



Conclusões

Essa aula foi muito interessante por conta da utilização do Python no Blender, já preferi muito mais do que fazer na mão, fora a praticidade de possuir tudo em código. Já consigo imaginar a utilização na criação de conjuntos de dados sintéticos, com variáveis como essa do Noise Scale recebendo valores aleatórios para gerar imagens diferentes sobre o objeto.

Nesse exemplo foi feito com o Noise Scale, mas facilmente pode ser algo como posição, tipo de objeto em um conjunto de X objetos, texturas, dentre inúmeras possibilidades.

Referencias

[Python Crash Course for Blender!](#)

[Blender 5.1 Python API Documentation](#)